TRUEAILAB Assignment – Production-Grade GenAI Assistant with RAG

Tech Stack (Mandatory)

- Backend: Python (FastAPI preferred)
- Frontend: Basic HTML/CSS/JavaScript
- Large Language Model API (OpenAI / Gemini / Claude / Mistral)
- Embeddings API
- RAG (Retrieval-Augmented Generation)
- Vector Storage (In-Memory, SQLite, ChromaDB, or FAISS)

Problem Statement

Build a production-style GenAI-powered Chat Assistant that answers user questions using Retrieval-Augmented Generation (RAG).

The assistant must:

- Convert documents into embeddings
- Store and retrieve them using vector similarity search
- Inject relevant context into the LLM prompt
- Generate grounded responses
- Maintain short conversation history

The solution must demonstrate real embedding-based retrieval. Keyword matching, hardcoded responses, or direct LLM calls without retrieval are not acceptable.

Functional Requirements

1. Document Knowledge Base

Students must create a `docs.json` file containing 5–10 meaningful documents.

Example

```
[
  {
    "title":"Reset Password",
    "content":"Users can reset their password from Settings > Security."
  }
]
```

Requirements

- Load documents from JSON
- Chunk long documents (300–500 tokens recommended)
- Generate embeddings for every chunk
- Store embeddings in vector storage
- Store chunk metadata:
    - document title
    - chunk id
    - source document

---

2. RAG Implementation (Core Requirement)

Students must:

During Indexing

- Load documents
- Chunk documents
- Generate embeddings
- Store vectors

During Querying

- Generate embedding for user query
- Perform similarity search
- Retrieve Top-K chunks (K = 3 recommended)
- Apply similarity threshold
- Build context from retrieved chunks
- Pass context to LLM

Grounding Requirements

If similarity score is below threshold:

- Return a safe fallback response

Example:

I could not find enough information in the knowledge base to answer this question.

Important

The implementation must use:

- Cosine Similarity OR
- Dot Product Similarity

Keyword search is not allowed.

---

3. LLM Integration

Students must integrate a real LLM API.

Supported providers:

- OpenAI
- Gemini
- Claude
- Mistral

Prompt Structure

Prompt must contain:

1. Retrieved Context
2. Conversation History
3. User Question

Example:

You are a helpful assistant.

Use ONLY the provided context to answer.

Context:
{retrieved_context}

Conversation History:
{history}

Question:
{user_question}

Requirements

- Temperature between 0 and 0.3
- Handle API failures
- Handle timeouts
- Handle invalid API keys
- Handle rate limits
- Log token usage (if available)

---

4. Chat Interface

Create a simple HTML-based frontend.

Required Features

- Message input
- Send button
- Chat display
- Loading indicator
- Session management using localStorage

Optional Features

- New Chat button
- Auto-scroll
- Markdown rendering
- Timestamps

---

5. Conversation Context

Students must maintain conversation memory.

Requirements

- Store last 3–5 message pairs
- Associate messages with sessionId
- Include history in prompt
- Ensure retrieved context remains the primary source of truth

---

6. API Requirements

Chat Endpoint

POST /api/chat

Request

```
{
  "sessionId":"abc123",
  "message":"How can I reset my password?"
}
```

Response

```
{
  "reply":"Users can reset their password from Settings > Security.",
  "tokensUsed":120,
  "retrievedChunks":3
}
```

Health Check Endpoint

GET /health

Response

```
{
  "status":"healthy"
}
```

Validation Requirements

Students must:

- Validate request payloads
- Return structured errors
- Log similarity scores
- Ensure retrieval happens before LLM invocation

Example Error

```
{
  "error":"Message field is required"
}
```

Expected Project Structure

```
project/
│
├── app/
│   ├── routes/
│   ├── services/
│   ├── models/
│   ├── vectorstore/
│   ├── prompts/
│   ├── utils/
│   └── main.py
│
├── frontend/
│   ├── index.html
│   ├── styles.css
│   └── app.js
│
├── docs.json
├── requirements.txt
├── .env.example
└── README.md
```

Deliverables

GitHub Repository containing:

- Complete source code
- README
- Setup instructions
- Architecture diagram
- RAG workflow explanation
- Embedding strategy explanation
- Similarity search explanation
- Prompt design reasoning

- Screenshots

Optional:

- Demo Video (5–10 minutes)

Evaluation Criteria

Area	Weight
RAG Architecture	30%
Embedding & Similarity Logic	25%
LLM Integration	20%
Prompt Design	10%
Frontend UI	5%
Code Quality & Error Handling	10%
Total	100%

Bonus Features (+20%)

Authentication

Implement JWT Authentication.

Persistent Storage

Store:

- Chat history
- Documents
- Embeddings metadata

Multi-Document Retrieval

Support answering questions across multiple documents.

Mandatory Submission Requirements

GitHub Repository

The complete source code must be pushed to a public GitHub repository.

The repository should include:

- Source code
- README.md
- requirements.txt
- .env.example
- Setup instructions

Live Deployment

The application must be deployed and accessible through a public URL.

Students may use any hosting platform, such as:

- Render
- Railway
- Vercel (Frontend)
- Netlify (Frontend)
- AWS
- Azure
- Google Cloud

The submitted repository must contain deployment instructions.

Submission Format

Students must submit:

GitHub Repository:
<https://github.com/><username>/<repository>

Live Application:
<https://your-app-url.com>